

Reviewer Pack — Discrepancy Lower Bound for Inner Product in High Dimensions

Entry 717e13caf4bb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 07:56:44 UTC

Discrepancy Lower Bound for Inner Product in High Dimensions

Entry ID: 717e13caf4bb **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 07:56:44 UTC

1. Conjecture

Field A (mathematical branch): DISCREPANCY_METHODS **Field B** (complexity object): COMMUNICATION_COMPLEXITY

Statement:

For any n -dimensional vector space over the reals, the discrepancy of the inner product function between two vectors is at least $\Omega(\sqrt{\log n})$ when the vectors are chosen from a certain set of unit vectors. This discrepancy lower bound implies that the communication complexity of computing the inner product is $\Omega(\sqrt{\log n})$ bits.

Rationale (proposer's reasoning):

This conjecture connects geometric discrepancy with communication complexity, leveraging dispersion methods to establish a lower bound. The discrepancy here measures how evenly the inner product values are distributed over the input space, and a lower bound on discrepancy would translate to a lower bound on the communication required to compute the function accurately.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f61438c9d7194404

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: INCONCLUSIVE against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - discrepancy methods communication complexity inner product lower bound - inner product discrepancy lower bound communication complexity - high dimensional inner product discrepancy communication complexity

Top relevant hits considered: - [<http://arxiv.org/abs/2307.07605v1>] First-order Methods for Affinely Constrained Composite Non-convex Non-smooth Problems: Lower Complexity Bound and Near-o - [<http://arxiv.org/abs/cs/0111062v2>] One-way communication complexity and the Neciporuk lower bound on formula size - [<http://arxiv.org/abs/cs/9906008v2>] A Lower Bound on the Average-Case Complexity of Shellsort - [<http://arxiv.org/abs/1112.2000v3>] A discrepancy lower bound for information complexity - [<http://arxiv.org/abs/2101.04525v2>] A comparison of optimisation algorithms for high-dimensional particle and astrophysics applications - [<http://arxiv.org/abs/2012.11098v2>] Sublinear Maximum Inner Product Search using Concomitants of Extreme Order Statistics - [<http://arxiv.org/abs/2401.14623v1>] Structure in Communication Complexity and Constant-Cost Complexity Classes

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=5.6s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    instances_tested = 5000
    discrepancy_sum = 0

    for _ in range(instances_tested):
        # Generate two random unit vectors
        u = [random.gauss(0, 1) for _ in range(n)]
        v = [random.gauss(0, 1) for _ in range(n)]

        # Normalize the vectors to make them unit vectors
        norm_u = math.sqrt(sum(x**2 for x in u))
        norm_v = math.sqrt(sum(x**2 for x in v))
        u = [x / norm_u for x in u]
        v = [x / norm_v for x in v]

        # Compute the inner product
        inner_product = sum(x * y for x, y in zip(u, v))
```

```

    # Accumulate the discrepancy
    discrepancy_sum += abs(inner_product)

# Calculate the mean discrepancy
mean_discrepancy = discrepancy_sum / instances_tested

# Calculate the lower bound
lower_bound = math.sqrt(math.log(n))

# Check if the conjecture holds
conjecture_holds = mean_discrepancy >= lower_bound

return {
    "metric_name": "discrepancy",
    "metric_value": mean_discrepancy,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"mean_discrepancy={mean_discrepancy}, lower_bound={lower_bound}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_discrepancy = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_discrepancy) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_discrepancy} std={std_dev} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_discrepancy} std={std_dev} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='{r['counterexample']}' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

sted': 5000, 'conjecture_holds': False, 'counterexample': 'mean_discrepancy=0.12789783157274617, lower_bound=0.12789783157274617'
TRIAL: {'metric_name': 'discrepancy', 'metric_value': 0.12793937463321467, 'instances_tested': 5000, 'conjecture_holds': False, 'counterexample': 'mean_discrepancy=0.12793937463321467, lower_bound=0.12793937463321467'}
TRIAL: {'metric_name': 'discrepancy', 'metric_value': 0.1277042472587447, 'instances_tested': 5000, 'conjecture_holds': False, 'counterexample': 'mean_discrepancy=0.1277042472587447, lower_bound=0.1277042472587447'}
TRIAL: {'metric_name': 'discrepancy', 'metric_value': 0.12827864896168484, 'instances_tested': 5000, 'conjecture_holds': False, 'counterexample': 'mean_discrepancy=0.12827864896168484, lower_bound=0.12827864896168484'}
TRIAL: {'metric_name': 'discrepancy', 'metric_value': 0.12792222855940152, 'instances_tested': 5000, 'conjecture_holds': False, 'counterexample': 'mean_discrepancy=0.12792222855940152, lower_bound=0.12792222855940152'}
TRIAL: {'metric_name': 'discrepancy', 'metric_value': 0.1261024102089387, 'instances_tested': 5000, 'conjecture_holds': False, 'counterexample': 'mean_discrepancy=0.1261024102089387, lower_bound=0.1261024102089387'}
TRIAL: {'metric_name': 'discrepancy', 'metric_value': 0.12550336206180757, 'instances_tested': 5000, 'conjecture_holds': False, 'counterexample': 'mean_discrepancy=0.12550336206180757, lower_bound=0.12550336206180757'}

```

TRIAL: {'metric_name': 'discrepancy', 'metric_value': 0.12699092074535773, 'instances_tested': 5000},
 TRIAL: {'metric_name': 'discrepancy', 'metric_value': 0.12437400121686437, 'instances_tested': 5000},
 TRIAL: {'metric_name': 'discrepancy', 'metric_value': 0.1262358236054102, 'instances_tested': 5000},

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before producing reliable results, preventing validation of conjecture status
 | next: Debug and rerun test with proper seed tracking implementation

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	117201
2	propose	ollama_remote	qwen3:8b	0	0	84934
3	preregistration	ollama_remote	qwen3:8b	0	0	28216
4	novelty	ollama_remote	qwen3:8b	0	0	22652
5	novelty	ollama_remote	qwen3:8b	0	0	27828
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11810
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7616
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7432
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9339
10	judge	ollama_remote	qwen3:8b	0	0	23407

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 340435 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/717e13caf4bb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/717e13caf4bb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/717e13caf4bb.tar.gz` (if generated)